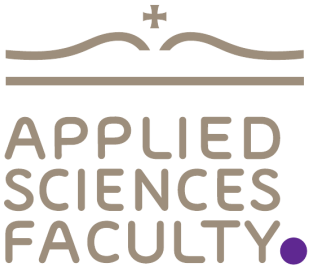
C++ — лекція 25. Атрибути та керування вирівнюванням





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Вирівнювання

- Коректне вирівнювання даних може бути критичним для отримання максимальної ефективності, або й взагалі — щоб мати можливість використовувати якусь апаратуру.
- Невирівняне звертання, наприклад, до int змушує процесор виконувати дві операції читання.
- SIMD-розширення часто потребують спеціального вирівнювання. Наприклад, дані для SSE[1] повинні бути вирівняними на границю 16 байт. Існують команди завантажувати із не-вирівняних адрес, але вони настільки повільніші від вирівняного доступу, що з'їдають практично весь виграш у продуктивності[2].
- Деякі процесори (в першу чергу – RISC із фіксованим розміром команд), вважаючи вказівники завжди вирівняними, економлять кілька біт у кодуванні команд та кількість ліній адреси.
- DMA-звертання часто потребують вирівняних адрес, як і всілякі memory-mapped I/O.
- Атомарні операції над не вирівняними даними, зазвичай, неможливі або неефективні.
- Невирівняні дані можуть перетинати межі рядка кешу чи сторінки, марно засмічуючи кеш чи TLB.

[1] Див. https://en.wikipedia.org/wiki/Streaming_SIMD_Extensions та, скажімо, http://sci.tuomastonteri.fi/programming/sse.

[2] Здається, на найновіших процесорах ця проблема пом'якшилася.

- Див. також обговорення тут: `https://stackoverflow.com/questions/381244/purpose-of-memory-alignment`.
  - Короткий огляд проблеми: `https://www.shincbm.com/embedded/2022/02/18/struct-layout-code-size.html`.
  - "Software Performance and Class Layout" – аналіз впливу на продуктивність.
- Компілятори в будь-якому випадку вирівнюють дані, так як їм видається ефективнішим для даної платформи. Однак, скажімо, на x86-32, int може бути вирівняним на границю 4-х байт, і коли потрібне вирівнювання на границю параграфу (16 байт), виникає проблема.
- Крім того, окремо є проблема вирівнювання динамічно виділеної пам'яті.
- Приклад, наскільки більше коду генерується для невирівняних даних на деяких RISC-процесорах: `https://devblogs.microsoft.com/oldnewthing/20200103-00/`, "Anybody who writes #pragma pack(1) may as well just wear a sign on their forehead that says 'I hate RISC' ".

Запит вирівнювання

- Компілятори надають ті чи інші нестандартні засоби керування вирівнюванням. Наприклад:

```
int x __attribute__ ((aligned (16))) = 0;     // GCC
__declspec( align( 16 ) ) int x = 0;          // MSVC
```

  Але нестандартні засоби, це, як мінімум — незручно.
- Нарешті, C++11 надає стандартний спосіб роботи із вирівнюванням.
- Оператор alignof повертає вирівнювання вказаного типу чи змінної.
- Аналогічно, структура alignment_of<T> містить статичну константу value, котра рівна вирівнюванню вказаного типу.
- Найменше вирівнювання у char[3] — 1.
- Тип std::max_align_t є синонімом типу із найбільшим вирівнюванням.
- Типи, для яких (програмістом) вимагається більше вирівнювання, ніж max_align_t, називають "over-aligned types а саму таку вимогу – "extended alignment". Підтримка таких типів мовними засобами — на розсуд реалізації. Детальніше див. en.cppreference.com/w/cpp/language/object#Alignment.

[3] Знаковий та беззнаковий варіант мають те ж вирівнювання.

Приклад:

```
struct Ratio { int num, den; };
struct Arr{ double ar[5]; };
struct Empty {};
struct alignas(64) Empty64 {};

    std::cout << "alignof(char) = " << alignof(char) << endl;
    std::cout << "alignof(int) = " << alignof(int) << endl;
    std::cout << "std::alignment_of<int>::value = "
              << std::alignment_of<int>::value << endl;
    std::cout << "alignof(long) = " << alignof(long) << endl;
    std::cout << "alignof(double) = " << alignof(double) << endl;
    std::cout << "alignof(long double) = " << alignof(long double) << end
    std::cout << "alignof(Ratio) = " << alignof(Ratio) << endl;
    std::cout << "alignof(Arr) = " << alignof(Arr) << endl;
    std::cout << "alignof(Empty) = " << alignof(Empty) << endl;
    std::cout << "alignof(Empty64) = " << alignof(Empty64) << endl;
    std::cout << "Maximal alignment: alignof(max_align_t) = "
              << alignof(max_align_t) << endl;
```

- MSVC 2015 не дозволяє використовувати його до змінних.

Керування вирівнюванням

- Оператор alignas задає вказане (в байтах) вирівнювання:

```
struct alignas(16) sse_float4_t{
    float data[4];
};
alignas(16) int x = 0;
alignas(128) char cacheline[128];
sse_float4_t f41, f42;                  // Aligned to 128 bytes
alignas(sse_float4_t) int i128;         // Aligned the same way
                                        // as sse_float4_t
```

- Може бути застосований до типу, змінної та при оголошенні класу/структури/об'єднання чи перерахування.
- Вирівнювання повинне бути степенем двійки. alignas(15) не скомпілюється.
- Вирівнювання, менші, ніж найбільше, вказане для об'єкту, ігноруються.
- Зауважте – якщо створити масив таких змінних, вирівняним буде кожен елемент.
- Вказання вирівнювання, меншого за природне, чи нав'язане іншими застосуваннями alignas, вважається помилкою. (Але і GCC і MSVC компілюють, останній – з попередженням, що вказівка була проігнорована). alignas(0) ігнорується завжди.

C

- C11 має ключові слова _Alignas та _Alignof, а alignas і alignof є макросами (існували до C23), які використовуються аналогічно до засобів C++. (В C++11 – це нові ключові слова)
- POSIX включає спеціальну функцію:

  void *memalign(size_t alignment, size_t size);

- Приклад, як викрутитися, коли безпосередньої підтримки вирівнювання немає: `https://embeddedartistry.com/blog/2017/2/20/implementing-aligned-malloc`.

std::align()

- Функція з <memory>:

```
void* align( std::size_t alignment,
             std::size_t size,
             void*& ptr,
             std::size_t& space );
```

  отримує:
  - ptr — вказівник на буфер, оновлюється функцією,
  - size – розмір елемента, який має бути вирівняним,
  - alignment — необхідне вирівнювання,
  - space — розмір буфера, оновлюється функцією.

  та повертає вказівник на вирівняний буфер, це ж значення потрапляє в ptr, підправляє space на кількість байт, що було використано для вирівнювання.
- Якщо вирівняти не вдалося – розмір замалий, ptr чи space не модифікує, повертає nullptr.
- Вирівнювання, не кратні 2 приводять до невизначеної поведінки[4].

[4] В C++17 ретроспективно виправлено помилку старіших стандартів, що також вважали невизначеною поведінкою вирівнювання, більші за alignof(max_align_t).

Приклад align()

- Наприклад:

```
char buffer [1024];
void* pbuf = buffer+1;
size_t bufsize = sizeof( buffer );
cout << "Buffer: " << (void*)buffer << ", bufsize " << bufsize
                                    << " -- Before." << endl;
char *newbuf = (char*)align(16, 128, pbuf, bufsize );
cout << "Buffer: " << (void*)pbuf << ", bufsize " << bufsize
                                    << " -- After." << endl;
```

Виведе:

```
Buffer: 0x22fa00, bufsize 1024 -- Before.
Buffer: 0x22fa10, bufsize 1009 -- After.
```

-

std::aligned_storage (C++11-C++20)

- Цей шаблон визнано застарілим в C++23, оскільки він створював більше проблем, ніж вирішував: `https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1413r3.pdf`.
- Рекомендована заміна:

```
- std::aligned_storage_t<sizeof(T), alignof(T)> t_buff;
+ alignas(T) std::byte t_buff[sizeof(T)];
```

- Шаблон із <type_traits>:

```
template<std::size_t Len, std::size_t Align = /*default-alignment*/>
struct aligned_storage;
```

містить тип type, котрий може служити неініціалізованим сховищем для об'єкту довжиною Len із вирівнюванням Align.

- Вирівнювання за замовчуванням — найбільше для типу розміру Len.
- Використовувати можна лише вирівнювання, які вже існують — Align = alignof(T) для якогось типу.

std::aligned_union (C++11-C++20)

- Цей шаблон визнано застарілим в C++23.
- Шаблон std::aligned_union робить те ж, що й std::aligned_storage, вибираючи найбільше із вирівнювань переданих типів:

```
template <size_t Len, class ... Types>
struct aligned_union;
```

- Порожній список типів приводить до невизначеної поведінки.
- Приклад std::aligned_storage та std::aligned_union:

```
struct Ratio {
    int num, den;
};

typedef std::aligned_storage<sizeof(Ratio),
                             alignof(max_align_t)>::type ratio_pod;
std::cout << "alignof(ratio_pod) = " <<  alignof(ratio_pod) << endl;

cout << "std::aligned_union demo: "
     << alignof(std::aligned_union<sizeof(Ratio), int, char, double>::type )
     << endl;
```

виведе (GCC 5.2, Mingw64):

```
alignof(ratio_pod) = 16
std::aligned_union demo: 8
```

C++17 and overaligned types

- C++17 додав деяку підтримку типів із "надмірним" вирівнюванням.
- Константа `__STDCPP_DEFAULT_NEW_ALIGNMENT__` містить вирівнювання динамічного виділення пам'яті по замовчуванню. Зараз в MSVC, GCC та Clang вона дорівнює 16.
- Якщо хотіти більше вирівнювання, компілятор використає один із нових new із явним вирівнюванням типу `std::align_val_t`:
- Приклад:

```
struct alignas(32) SIMD_data {
    double a, b, c;
};
    ..................
auto p = new SIMD_data[10];
```

Зауважте, буде виділено блок пам'яттю 32*10 байт.

- Якщо потрібно вирівняти динамічну пам'ять не так, як вказано alignas(), можна спробувати зробити це безпосередньо:

  auto p = **new**(std::align_val_t{ 64 }) SIMD_data;

- Однак, виникає проблема – delete по замовчуванню не підходитиме, він не знає про таке "runtime-задане" вирівнювання, а можливості передати аргументи немає. Тому доведеться викликати деструктор і оператор виділення вручну:

  p->~SIMD_data();
  ::**operator delete**(p, std::align_val_t{64});

- Тому, **краще так просто не робити**.
- Дискусію на тему див. тут `https://developercommunity.visualstudio.com/content/problem/528320/using-c17-new-stdalign-val-tn-syntax-results-in-er.html`.

std::assume_aligned (C++20)

- Функція:

  template< std::size_t N, **class** T >
  [[nodiscard]] constexpr T* assume_aligned(T* ptr);

  повідомляє компілятору, що об'єкт є вирівняним на N – це може бути використано для оптимізації.
- N – степінь двійки.
- Щоб компілятор міг скористатися цією інформацією, до відповідного об'єкта потрібно звертатися за допомогою вказівника, що повернула ця функція.

Посилання

- "Anybody who writes #pragma pack(1) may as well just wear a sign on their forehead that says "I hate RISC"". Зокрема, містить приклади асемблерного коду для вирівняних та не вирівняних звертань для купи архітектур.
- "Use source-level annotations to help GCC detect buffer overflows" – приклад атрибутів анотації буферів, що передаються функціям:

```
__attribute__ ((access (write_only, 2, 3))) ssize_t
read (int fd, void *buf, size_t nbytes);
__attribute__ ((access (read_only, 2, 3))) ssize_t
write (int fd, const void *buf, size_t nbytes);
```

- API для WIndows керувати вирівнюванням і локалізацією адрес: "How to allocate address space with a custom alignment or in a custom address region".
- "How can I tell C++ that I want to discard a nodiscard value?".
- "Member ordering and binary sizes" – есей про вплив вирівнювання на розмір бінарних файлів. Цікаво, що якщо елементи нульові – не впливатиме (BSS).
- "How to compress out interior padding in a std::pair and why you don't want to".
- "Why do some Windows functions fail if I pass an unaligned Unicode string?" – Якщо вказівник на тип T не вирівняний згідно вимог типу T, в C це невизначена поведінка, в C++ трохи простіше – unspecified.
- "The Hidden Cost of Misalignment" на прикладі RISC-V. Показано цікавий інструмент: `__attribute__((packed, aligned(4)))` – структура запакована, але в пам'яті вирівняна.
- "Investigating Split Locks on x86-64" – про катастрофічні наслідки для продуктивності атомарних операцій, що перетинають рядок кешу.

Атрибути

- Є багато речей, які залежать від реалізації. Компілятори викручуються, як можуть...

  \\ MSVC:
  __declspec(dllexport) **void** fn(){}
  \\ GCC:
  __attribute__ ((dllexport)) **void** fn(){}

- В C++11 з'явився стандартизований інтерфейс для таких розширень мови:

  struct [[someattribute]] special_data_t { };

- Атрибути можуть стосуватися практично чого завгодно в коді.
- При оголошенні атрибути можуть йти перед іменем або зразу після нього (такі атрибути діятимуть одночасно), у інших випадках – безпосередньо перед тим, на що діють.
- Невідомі директиви ігноруються.
- Може бути перераховано декілька, через кому: `[[attr1, attr2]]`.
- Атрибути можуть бути в своїх просторах імен та можуть мати аргументи: `[[namespace::attr(args)]]`.
- Така собі модернова pragma...

Стандартні атрибути C++11

Стандарт оголошує лише наступні атрибути:

- `[[noreturn]]` — функція не завершується. (Якщо вона таки завершиться – невизначена поведінка). Стосується функцій типу abort(), функцій із безмежними циклами та таких, що завжди кидають виключення[5].
- `[[carries_dependency]]` — детальніше див. лекцію про багатопоточність в C++11.
- `[[deprecated]]` та `[[deprecated("reason")]]` в C++14 — вказує давати попередження при використанні помічених так сутностей (функцій, класів, і т.д. і т.п.).
- Детальніше див. `http://en.cppreference.com/w/cpp/language/attributes` та, скажімо, `http://www.codesynthesis.com/~boris/blog/2012/04/18/cxx11-generalized-attributes/` і `https://medium.com/@sshambir/как-пользоваться-атрибутами-из-c-17-622a6b53225a`.

[5] Приклад: `https://quuxplusone.github.io/blog/2022/06/29/that-undiscovered-country/`.

C++17

- Додано using-форму, тепер можна писати:
  `[[using rpr: kernel, target(cpu,gpu)]]` замість:
  `[[ rpr::kernel, rpr::target(cpu,gpu) ]]`. Змішувати не можна!
  - Приклад: [[clang::musttail]]:
    `https://clang.llvm.org/docs/AttributeReference.html#musttail`.
  - Приклад: [[gnu::flatten]]
    `https://gcc.gnu.org/onlinedocs/gcc/Common-Function-Attributes.html`.
- `[[fallthrough]]` — для switch, щоб не було попереджень про "провалювання"до іншої мітки.
- `[[nodiscard]]` — застосовується до функцій, перерахувань, класів та структур, спричиняє попередження, якщо результат функції чи повернена змінна такого типу проігнорована.
  - `https://quuxplusone.github.io/blog/2023/08/11/style-guide-guide/` – на прикладі nodiscard та аналогів, хороша стаття про те, що "style guide" має описувати високорівневу концепцію, а не конкретні синтаксичні способи її досягнення.
- `[[maybe_unused]]` — навпаки, вимикає попередження про невикористаний параметр чи змінну.

C++20

- `[[likely]]/[[unlikely]]` – більш/менш ймовірна гілка if(), switch() тощо.
- `[[no_unique_address]]` – дозволяє об'єктам не мати унікальної адреси, дозволяє оптимізувати, викидаючи порожні члени класів.
- `[[nodiscard]]` тепер можна застосовувати і до конструкторів – на випадок, коли створеній змінній забули надати ім'я абощо.
  - `[[nodiscard("Message")]]` – тепер можна передавати повідомлення.
- Детальніше див. `http://en.cppreference.com/w/cpp/language/attributes`, `https://www.cppstories.com/2021/no-unique-address/` та, скажімо, `http://www.codesynthesis.com/~boris/blog/2012/04/18/cxx11-generalized-attributes/`.

C++23

- `[[assume(expression)]]` – дуже цікавий атрибут[6]!

```
void f(int x, int y) {
    [[assume(x > 0)]];
        .....................
    x = 3;
    int z = x;
    [[assume((h(), x == z))]]; // Вважати, що після виклику функції h(),
                               // x буде дорівнювати z.
                               // Виклик функції не буде відбуватися.
    h();                       // Стосуватиметься саме цього виклику, не наступних
```

  - Приклад з `https://en.cppreference.com/w/cpp/language/attributes/assume`.
  - Порушення умови – невизначена поведінка.
  - Тому не потрібно зловживати – це лише підказка оптимізатору!
- Дозволено додавати атрибути до лямбда-функцій:

```
auto lm = [][[nodiscard]]()->int { return 42; };
```

- Також, дозволили повтори атрибутів у списку.
- `https://www.sandordargo.com/blog/2022/12/14/cpp23-attributes`.

[6] Нагадує мені аналогічні конструкції з систем комп'ютерної алгебри – певний інтеграл може мати різні формули для різних знаків котрогось з коефіцієнтів.

__has_cpp_attribute(attribute-token)

- `__has_cpp_attribute( attribute-token )` дозволяє перевірити наявність атрибута.
- Розгортається в 0, якщо атрибута немає, в рік-місяць його включення в стандарт для стандартних атрибутів та якесь ненульове значення для нестандартних.
- Приклад:

```
#if !__has_cpp_attribute(noreturn)
#error "Need noreturn!"
#endif
```

Приклад:

- ```
  [[ noreturn ]] void fn() {
    throw "error";
  }

  struct [[nodiscard]] error_info { };

  [[maybe_unused]] void fn2( [[maybe_unused]] int a,
                             [[maybe_unused]] int b)
  {
      [[maybe_unused]] int c = a + b;
  }
  ```
- Із switch:

  ```
  switch(responce)
  {
  case 'a':
       action_for_a();
       break;
  case 'x':
            [[ fallthrough ]];
  case 'X':
       exit_program=true;
       break; // Завершуємо switch
  };
  ```
  ```

C++26

- `[[indeterminate]]` – підказка, що функція лише записує у свій аргумент, і це OK, якщо він не ініціалізований ("Erroneous behaviour for uninitialized reads"):

```
int x [[ indeterminate ]];
      std :: cin >> x;
```

- Тепер можна використовувати атрибути в розпакуванні:

```
auto [a, b [[maybe_unused]], c] = f().
```

Нестандартні атрибути

- GCC та GLANG підтримують таку форму своїх атрибутів: `[[gnu::attribute]]`. Скажімо, так можна позбутися попереджень про не використані значення a, b, c.

```
[[gnu::unused]] void fn3( [[gnu::unused]] int a,
                          [[gnu::unused]] int b)
{
    [[gnu::unused]] int c = a + b;
}
```

- Список атрибутів функцій в GCC: `https://gcc.gnu.org/onlinedocs/gcc/Function-Attributes.html`.
- Список атрибутів змінних в GCC: `https://gcc.gnu.org/onlinedocs/gcc/Variable-Attributes.html`.
- ◇ `__attribute__((optimize("-ffast-math")))` – зокрема, цікавий атрибут в GCC. В Clang є схожі інструменти.

Посилання

- "How can I tell C++ that I want to discard a nodiscard value?".
- "On the Ignorability of Attributes" – есей про проблеми з атрибутами.

C++26 – підтримка відлагодження, <debugging>

- std::breakpoint() – перервати виконання програми та передати керування дебагеру.
  - ◇ Системозалежна – аналог `asm("int3")` на x86, __debugbreak з WinAPI – яка запропонує запустити дебагер тощо.
- std::breakpoint_if_debugging() – передати керування дебагеру, якщо він активний.
- std::is_debugger_present() – перевірити, чи дебагер присутній.

Stacktrace – C++23

- Бібліотека <stacktrace> дозволяє отримати доступ до, власне, стеку викликів.
- Через важливість питання виділення пам'яті в таких ситуаціях, std::stacktrace – синонім для std::basic_stacktrace з алокатором за замовчуванням.
- Для отримання стеку викликів потрібно скористатися статичною функцією std::stacktrace::current():
- Існує в трьох варіантах: отримати весь стек викликів; його частину, починаючи з певного індексу; або – починаючи з індексу і не більше певної глибини.
- Отриманий об'єкт можна надрукувати:

```
auto trace = std::stacktrace::current();
std::cout << std::to_string(trace) << std::endl;
```

- Якщо потрібен детальніший доступ – він є контейнером, по якому можна ітерувати:

```
auto trace = std::stacktrace::current();
for (const auto& entry: trace) {
    std::cout << "Desc: " << entry.description() << std::endl;
    std::cout << "file: " << entry.source_file() << std::endl;
    std::cout << "line: " << entry.source_line() << std::endl;
}
```

◇ Приклад взято тут: `https://www.sandordargo.com/blog/2022/09/21/cpp23-stacktrace-library`.

- А ось статичну функцію from_current_exception() поки не стандартизували – планується в C++26.
- Під GCC 12+ потребує `-lstdc++_libbacktrace`.
- Див. також "A Proposal to add stacktrace library".
- "Improve Diagnostics with std" – есей на тему.



